

Final Exam Practice Questions and Hints

Stream Processing and Differential Privacy

开卷

COMPCSI 752: Big Data Management

University of Auckland

Karuna Reddy

Auckland, June 2025



Question 1 – S1 2024

MapReduce (10 marks)

The input contains two tables. Table T_1 contains voter information for each voter: (voter-id, name, age, zip). Table T_2 contains disease information: (age, zip, disease). For each unique pair of {age, zip} values, the output should contain a list of names and a list of diseases for the voters in {age, zip}. If a particular {age, zip} pair appears in one table but not the other, then you omit it from the output entirely.

Example:

Table T_1 with 3 records: [ID-1, A, 18, 1000], [ID-2, B, 19, 1000], [ID-3, C, 18, 1000],

Table T_2 with 3 records: [18, 1000, X], [19, 1001, Y], [18, 1000, Z],

The output will be: {(18, 1000), [(A, C), (X, Z)]}

Write a MapReduce function to solve the question above. For Map() and Reduce(), indicate the key and value of both input and output.



Solution Q1

We are given two tables: **T1**: (voter-id, name, age, zip); and **T2**: (age, zip, disease)

We must **join** on (age, zip) and output: {(age, zip): ([names], [diseases])}

Only if the pair exists in **both** T1 and T2.

Map(record)

Input:

- **Key**: irrelevant (e.g., record ID or row number)
- **Value**: a full record from either T1 or T2

Output:

- **Key**: (age, zip)
- **Value**: ("NAME", name) if from T1
("DISEASE", disease) if from T2

Reduce(key, values)

Input:

- **Key**: (age, zip)
- **Value**: A list of values tagged as either "NAME" or "DISEASE"

Output:

- **Key**: (age, zip)
 - **Value**: ([names], [diseases])
- Only emit if both lists are non-empty



Solution Q1

```
def Map(record):  
    if record.source == "T1":  
        # T1: (voter-id, name, age, zip)  
        key = (record.age, record.zip)  
        value = ("NAME", record.name)  
        Emit(key, value)  
  
    elif record.source == "T2":  
        # T2: (age, zip, disease)  
        key = (record.age, record.zip)  
        value = ("DISEASE", record.disease)  
        Emit(key, value)
```

```
def Reduce(key, values):  
    names = []  
    diseases = []  
  
    for vtype, val in values:  
        if vtype == "NAME":  
            names.append(val)  
        elif vtype == "DISEASE":  
            diseases.append(val)  
  
    # Emit only if we have both names  
    if names and diseases:  
        Emit(key, (names, diseases))
```

不要手写代码。



Solution - Map & Reduce Steps

MAP

For T1 (voter info), we emit:

key = (record.age, record.zip)

value = ("NAME", record.name)

And for T2 (disease info), we emit:

key = (record.age, record.zip)

value = ("DISEASE", record.disease)

REDUCE

Emit(key, (names, diseases))



Solution Q1

For Example:



For **input** records:

T1: [ID-1, A, 18, 1000], [ID-2, B, 19, 1000], [ID-3, C, 18, 1000]

T2: [18, 1000, X], [19, 1001, Y], [18, 1000, Z]

Only (18, 1000) is common in both tables.

Output:

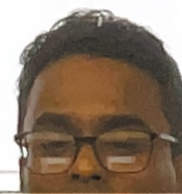
((18, 1000), (["A", "C"], ["X", "Z"])))



Question 2

Differential Privacy (15 marks)

- (a) Explain how the released data in Table T_1 in Question 5 breach the *differential privacy* given the output of the MapReduce function? [5 marks]
- (b) Consider the **new query** Q : $f(T_1)$ = average age of the voters in T_1 . Assume that the age of any voter in T_1 ranges between 20 and 100. Consider a mechanism $\mathcal{M}$ obtained by adding Laplacian noise $Lap(b)$ to the answer of Q , i.e. releasing $f(T_1) + Lap(b)$ where b is a parameter.
- Given that T_1 has 1,000 voters, does $\mathcal{M}$ satisfy ϵ -differential privacy for some ϵ given the output of the query Q ? If the answer is yes, give the minimum value of such ϵ . If the answer is no, give a counterexample. [5 marks]
- (c) How could we use the mechanism $\mathcal{M}$ to ensure ϵ -differential privacy for the output of the MapReduce function in Question 5? Explain your answer. [5 marks]



Solution 2 (a)

Linkage via Join Key:

The MapReduce function joins T1 and T2 on **(age, zip)**, a quasi-identifier that may uniquely identify individuals.

Names + Diseases Linked:

Since T1 reveals names and T2 reveals diseases, the output exposes **which people are linked to which diseases**, violating privacy.

Inference from Output:

If (18, 1000) only appears once with name “A” and diseases X, Z — an observer can infer that “A” likely has disease X or Z.

Fails Differential Privacy Goal:

The output changes significantly based on **who is present or absent**, allowing personal data inference — which violates differential privacy guarantees.

Solution 2 (b)

Given:

- Query $Q = \text{average age of voters in table } T1$
- Each age $\in [20, 100]$
- Table $T1$ has $n = 1000$ voters
- Mechanism:

$$M(T1) = \text{average}(T1) + \text{Lap}(b)$$

We are asked:

Does this satisfy **ϵ -differential privacy** for some ϵ ?

If yes, find **minimum ϵ** such that it holds.

If no, give a counterexample.



Solution 2 (b)

Step 1: Understand the sensitivity of the query

To determine if a Laplace mechanism satisfies ϵ -differential privacy, we need the **L1-sensitivity** of the query.

The **sensitivity** Δf is the maximum change in the function's value when one individual's data is changed (added, removed, or replaced).

Here, the function is the **average** over 1000 values in T1.

If one person's age is changed from the minimum (20) to the maximum (100), the average can change by:

$$\Delta f = \frac{|100 - 20|}{1000} = \frac{80}{1000} = 0.08$$

Step 2: Use Laplace mechanism

We're adding Laplace noise with scale parameter **b**.

The **Laplace mechanism** guarantees ϵ -differential privacy if:

$$b = \frac{\Delta f}{\epsilon} \Rightarrow \epsilon = \frac{\Delta f}{b} = \frac{0.08}{b}$$

Solution 2 (c)

1. Aggregate Instead of Releasing Raw Data

Replace lists of names/diseases with aggregate counts or summaries per (age, zip) group.

2. Add Laplace Noise to Counts

Use the **Laplace mechanism**: for each count, add noise scaled to sensitivity ($\Delta f = 1$), using noisy count = true count + $\text{Lap}(1/\epsilon)$

3. Suppress Low-Support Outputs

Only emit outputs where noisy counts exceed a threshold — this reduces risk from small groups.

4. Avoid Direct Linkage

Do not emit raw names or diseases. Use general categories or anonymized encodings to protect identity linkage.

Question 3 - Data Stream – S1 2023

Data stream - Bloom filter

[10 marks]

Suppose we have n bits of memory available and a set S has m elements. Instead of using k hash functions each mapping an element to a bit in the memory, we could divide the n bits into k subarrays (assume n is divisible by k), and then use the i -th hash function, $i \in [1..k]$, to the i -th subarray.

- (a) As a function of n , m , and k , what is the probability of a false positive? [5 marks]
- (b) How does it compare with the standard setting of Bloom filter that uses k hash functions into a single array? [5 marks]

Solution – Q3 (a)

- Total memory: n bits
- Hash functions: k
- Each subarray: n/k bits
- Number of inserted elements: m

We assume:

- Each hash function only touches its own subarray (i.e., hash h_i writes to subarray i).

So for each subarray of size n/k , the probability that a given bit is still 0 after inserting m elements is:

$$\left(1 - \frac{1}{n/k}\right)^m = \left(1 - \frac{k}{n}\right)^m$$

So the probability a bit is 1 in that subarray is:

$$1 - \left(1 - \frac{k}{n}\right)^m$$

Since a false positive means all k bits (one from each subarray) are 1:

$$\left[1 - \left(1 - \frac{k}{n}\right)^m\right]^k$$

Solution – Q3 (b)

In a **standard Bloom filter**, all k hash functions write into a **shared array of size n** .

In contrast, in this question's variant, **each function writes into its own subarray** (each of size n/k).

Differences in filtering performance:

- **False Positive Rate (FPR):**

- **Standard Bloom Filter:** $\text{FPR} \approx (1 - e^{-km/n})^k$ (optimized for joint usage of bits)

- **Subarray Variant** (this question): $\text{FPR} = \left[1 - \left(1 - \frac{k}{n}\right)^m\right]^k$

- **Implication:**

- Standard filter uses memory more flexibly — **all k functions get to use the full array**, leading to a **lower FPR**.
- The subarray method effectively reduces memory per hash function → **higher FPR** for same n, m, k .
- The subarray approach is simpler to implement (independent subarrays), but **less efficient** in practice.

Question 4 – S1 2022

Data stream

[10 marks]

In our lecture, we have studied the reservoir sampling which samples an element from a stream of size m with the same probability. If we use the reservoir sampling with the summary size $s = 1$, each element of a stream will be sampled with probability $1/m$. We name this method as RS_1 . The generalized version of reservoir sampling with the summary size $s > 1$ guarantees that each element in a stream will be sampled with the same probability s/m . We name this method as RS_s .

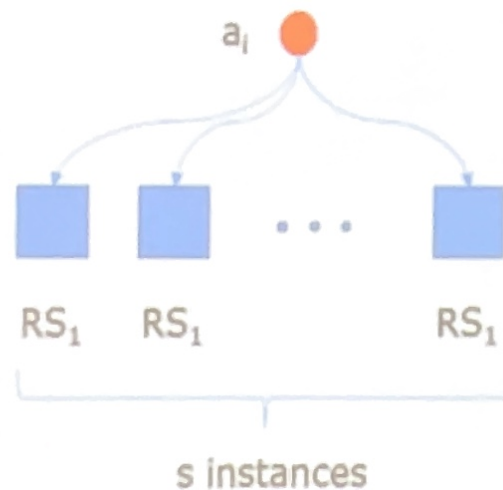


Figure 5: Illustration of how pRS_1 works.

In the exam, we consider a new algorithm, called pRS_1 , that simulates RS_s for $s > 1$ by running s independent RS_1 instances in parallel. pRS_1 also uses a summary of size s , as shown in Figure 5.

- (a) As a function of m and s , what is the probability an element of a stream is sampled by pRS_1 ? [8 marks]
- (b) Let f_i be the number of occurrences of the element a_i in a stream. Explain how we can use RS_s and pRS_1 to estimate f_i . [2 marks]

Solution Q4 (a)

Let:

- m : total number of elements in stream
- s : number of independent RS_1 instances

Step 1:

Probability that an element is **not** selected by one instance of RS_1 :

$$1 - \frac{1}{m}$$

Step 2:

Probability that an element is **not** selected by any of the s instances:

$$\left(1 - \frac{1}{m}\right)^s$$

Step 3:

Probability that the element is selected by **at least one** instance:

$$\Pr[\text{sampled by } pRS_1] = 1 - \left(1 - \frac{1}{m}\right)^s$$

Question on Data Stream

Applied question on Data Stream worth **17 marks**.

Format:

Part (a) (i) 3 Marks, (ii) 2 Marks

(b) (i) 2 Marks, (ii) 3 Marks

(c) (i) 3 Marks, (ii) 4 Marks

bloom filter.
如何保持误差.

sampling hashing.

作业的算法.
M-G.

Question on Differential Privacy

Applied question - Differential Privacy worth **17 Marks**.

Format:

explain
which better.

Part (a) 6 Marks

平均能听到泄露信息

(b) (i) 3 Marks, (ii) 3 Marks

(c) (i) 2 Marks, (ii) 3 Marks

Laplace...

map reduce

7/17



